Normalise audio

You can increase or decrease the volume level in an audio or video file using the volume filter. You set the filter value to a multiple of the input volume or specify the maximum loudness level in decibels.

```
ffmpeg -i low.mp3 -af 'volume=3' high.
mp3
```

Adjusting the volume in this manner may not work all the time. Sometimes, the audio is so low that even doubling it does not make a difference. The correct solution involves two steps. First, use the volumedetect filter to determine the loudness of audio samples in the file. Then, apply the volume filter based on what you learnt in the first step.

```
ffmpeg -i low.mp3 -af "volumedetect" -f
null -
```

```
[Parsed_volumedetect_0 @ 0x226c100] mean_volume: -32.4 dB
[Parsed_volumedetect_0 @ 0x226c100] max_volume: -17.3 dB
[Parsed_volumedetect_0 @ 0x226c100] histogram_17db: 6
[Parsed_volumedetect_0 @ 0x226c100] histogram_18db: 15
[Parsed_volumedetect_0 @ 0x226c100] histogram_19db: 56
[Parsed_volumedetect_0 @ 0x226c100] histogram_20db: 452
[Parsed_volumedetect_0 @ 0x226c100] histogram_21db: 1676
```

Figure 6: Use volumedetect filter before using volume filter

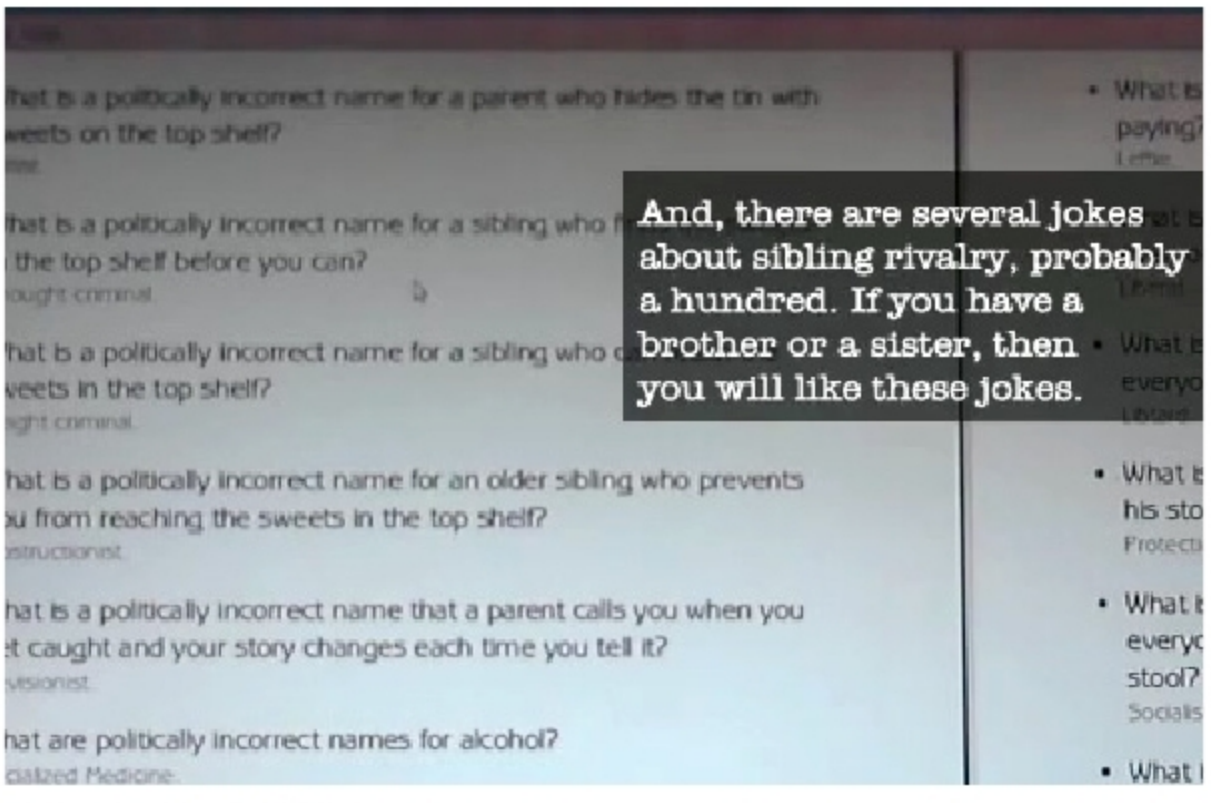

Figure 7: A video with burned-in subtitles

In Figure 6, the volumedetect filter shows that maximum loudness of all samples is at 17dB. The histogram shows the number of samples that would get clipped if the volume was taken any higher than 17. For example, increasing the volume to 18dB would clip the waveform in six samples.

After studying this, it is clear that the loudness needs to be raised to a maximum of 17dB.

```
ffmpeg -i low.mp3 \
       -af 'volume=17dB' -f ogg \
       normalized.ogg
```

Normalisation preserves the original waveform while increasing only the loudness.

Slipstream audio to a video

Sometimes, I encounter a video where the audio has some problems. I usually extract the audio stream to an MP3 file so that I can examine it in Audacity. After I fix the problem, I export the corrected audio to another MP3 file. Then, I remove the original audio from the video and add this corrected MP3 stream as its audio. I use *maps* to accomplish this.

```
#Extract the audio
ffmpeg -i original-video.mp4 \
       -map 0:1 \
       problem-audio.mp3

# Fix the problem in problem-audio.mp3
using Audacity and create
# corrected-audio.mp3

#Replace existing audio with fixed audio
from mp3
ffmpeg -i original-video.mp4 -i
corrected-audio.mp3 \
       -map 0:0 -map 1:0 \
       -codec copy \
       video-with-corrected-audio.mp4
```

In the first command, *-map 0:1* refers to the first input file's or the video's second stream, which happens to be the audio stream. (The *-map* option may be redundant but it helps in debugging more complex commands.) In the second *ffmpeg* command, *-map 0:0* refers to the first input file's or the video file's first stream, which is the video stream. The map in *-map 1:0* refers to the second input file's or the audio file's first stream, which is the corrected audio stream.

Slipstream subtitles to video

Subtitles were very useful to me when I had to create a book-read video for my first book. The audio had issues and I decided to create subtitles and burn them into the video. (I used the GNOME subtitles program to transcribe the video and save to a subtitle file. It has simple keyboard shortcuts for controlling the playback [play/pause/rewind/fast forward] of the video while typing the subtitles. No need to use the mouse.)